

Transaction Management

3.4 Making Use of Transactions

3.4.1 Conditionally Committing or Rolling Back

Objective: Learn how to conditionally commit or roll back transactions based on specific conditions.

Example 1: Conditionally Committing a Transaction

```
BEGIN TRANSACTION;
```

```
DECLARE @TransactionSuccess BIT;
```

```
SET @TransactionSuccess = 1;
```

```
-- Perform some operations
```

```
UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
```

```
IF @@ERROR <> 0 SET @TransactionSuccess = 0;
```

```
UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
```

```
IF @@ERROR <> 0 SET @TransactionSuccess = 0;
```

```
-- Check condition and commit or rollback
```

```
IF @TransactionSuccess = 1
```

```
    COMMIT TRANSACTION;
```

```
ELSE
```

```
    ROLLBACK TRANSACTION;
```

Example 2: Conditionally Rolling Back a Transaction with Error Message

```
BEGIN TRANSACTION;
BEGIN TRY
    -- Perform some operations
    UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
    UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
    -- Commit the transaction if no errors
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Rollback the transaction in case of error
    ROLLBACK TRANSACTION;
    PRINT 'Error occurred, transaction rolled back';
END CATCH;
```

3.4.2 Transactions with Error Handlers

Objective: Implement transactions with advanced error handling to ensure data integrity.

Example 1: Using TRY...CATCH for Error Handling

```
BEGIN TRANSACTION;
BEGIN TRY
    -- Perform some operations
    INSERT INTO Orders (CustomerID, OrderDate) VALUES (1, GETDATE());
    INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(SCOPE_IDENTITY(), 1, 10);
    -- Commit the transaction if no errors
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Rollback the transaction in case of error
    ROLLBACK TRANSACTION;
    PRINT 'Error occurred, transaction rolled back: ' + ERROR_MESSAGE();
END CATCH;
```

Example 2: Nested TRY...CATCH for Detailed Error Handling

```
BEGIN TRANSACTION;
BEGIN TRY
    BEGIN TRY
        -- Perform some operations
        INSERT INTO Orders (CustomerID, OrderDate) VALUES (1, GETDATE());
    END TRY
    BEGIN CATCH
        -- Handle errors for Orders table
        PRINT 'Error occurred in Orders table: ' + ERROR_MESSAGE();
        THROW;
    END CATCH;

    BEGIN TRY
        -- Perform some operations
        INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES
(SCOPE_IDENTITY(), 1, 10);
    END TRY
    BEGIN CATCH
        -- Handle errors for OrderDetails table
        PRINT 'Error occurred in OrderDetails table: ' + ERROR_MESSAGE();
        THROW;
    END CATCH;

    -- Commit the transaction if no errors
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Rollback the transaction in case of error
    ROLLBACK TRANSACTION;
    PRINT 'Transaction rolled back due to errors';
END CATCH;
```

3.4.3 Automatic Rollback

Objective: Implement automatic rollback in transactions to ensure data consistency.

Example 1: Automatic Rollback with SET XACT_ABORT

```
SET XACT_ABORT ON;
BEGIN TRANSACTION;
BEGIN TRY
    -- Perform some operations
    UPDATE Products SET Quantity = Quantity - 10 WHERE ProductID = 1;
    -- Intentionally cause an error
    INSERT INTO Products (ProductID, ProductName) VALUES (1, 'Duplicate Product');
    -- Commit the transaction
    COMMIT TRANSACTION
END TRY
BEGIN CATCH
    -- Automatically rollback the transaction
    PRINT 'Error occurred, transaction rolled back: ' + ERROR_MESSAGE();
END CATCH;
```

Example 2: Ensuring Automatic Rollback in Complex Transactions

```
SET XACT_ABORT ON;
BEGIN TRANSACTION;
BEGIN TRY
    -- Perform multiple operations
    DELETE FROM Inventory WHERE ProductID = 1;
    INSERT INTO Inventory (ProductID, Quantity) VALUES (1, 50);
    -- Commit the transaction
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Automatically rollback the transaction
    PRINT 'Transaction rolled back due to error: ' + ERROR_MESSAGE();
END CATCH;
```

Nested Transactions

Objective: Learn to manage nested transactions and ensure proper rollback or commit in nested scenarios.

Example 1: Simple Nested Transactions

```
BEGIN TRANSACTION;
BEGIN TRY
    -- Outer transaction
    UPDATE Customers SET Status = 'Active' WHERE CustomerID = 1;
    BEGIN TRANSACTION;
    -- Inner transaction
    BEGIN TRY
        INSERT INTO Orders (CustomerID, OrderDate) VALUES (1, GETDATE());
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- Rollback inner transaction
        ROLLBACK TRANSACTION;
        PRINT 'Inner transaction rolled back';
        THROW;
    END CATCH;
    -- Commit outer transaction
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Rollback outer transaction
    ROLLBACK TRANSACTION;
    PRINT 'Outer transaction rolled back: ' + ERROR_MESSAGE();
END CATCH;
```

Example 2: Nested Transactions with Save points

```
BEGIN TRANSACTION;
BEGIN TRY
    -- Outer transaction
    UPDATE Products SET Quantity = Quantity - 10 WHERE ProductID = 1;
    -- Savepoint
    SAVE TRANSACTION SavePoint1;
    BEGIN TRY
        -- Inner transaction
        INSERT INTO Inventory (ProductID, Quantity) VALUES (1, 50);
        -- Commit only if no errors
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- Rollback to savepoint on error
        ROLLBACK TRANSACTION SavePoint1;
        PRINT 'Inner transaction rolled back to SavePoint1';
        THROW;
    END CATCH;
END TRY
BEGIN CATCH
    -- Rollback outer transaction
    ROLLBACK TRANSACTION;
    PRINT 'Outer transaction rolled back: ' +
```